

```

"scripts": {
  "start": "node ./src/index.js"
},
"author": "Krishna Mohan Koyya",
"license": "ISC",
"dependencies": {
  "body-parser": "^1.19.0",
  "cors": "^2.8.5",
  "express": "^4.17.1",
  "mysql2": "^2.3.3",
  "request": "^2.88.2"
}
}

```

As evident from the above listing, we are depending on the *express* library for web interfacing, *request* library for HTTP request processing, *cors* library for handling cross-origin scripting, *body-parser* library for parsing the HTTP payloads, and *mysql2* library for connecting to the MySQL database. These libraries are not part of Node. The following command gets them for us.

```
npm install
```

This completes the project setup. Now, let us move on to coding. Create a folder named *src* in the project root. This folder holds the source code for all the layers of the design.

The domain layer

The layer of the core domain consists of *Name*, *PhoneNumber*, *User* and *UserRepository*. A decade ago, implementing object-oriented design using JavaScript was rather challenging. Not any more. ECMAScript, or ES in short, has made the job a lot easier.

Given below is the *Name* class. The constructor accepts the name string and validates it before assigning it to the private attribute *value*. Since *Name* is a value object, it maintains the immutability of the object, once created. In other words, it does not offer any setters.

```

class Name{
  #value;
  constructor(value){
    if (value == null || value.trim().length < 8 ||
value.trim().length > 32)
      throw "Invalid Name";
    this.#value = value;
  }

  getValue(){
    return this.#value;
  }
}

```

```

equals(other){
  if(this.constructor.name === other.constructor.name
&& this.#value === other.getValue())
    return true;
  return false;
}

export default Name;

```

It's the same case with the *PhoneNumber* as well, since it is also a value object.

```

class PhoneNumber {
  #value;

  constructor(val){
    let matched = String(val).match(/\d/g).length===10;
    if(!matched)
      throw "Invalid Phone Number";
    this.#value = val;
  }

  getValue(){
    return this.#value;
  }

  equals(other){
    if(this.constructor.name === other.constructor.name
&& this.#value === other.getValue())
      return true;
    return false;
  }
}

export default PhoneNumber;

```

The *User* is an entity. Since the *SearchService* only retrieves the existing user details, it makes sense to offer a constructor that sets the full state.

```

class User{
  #name;
  #phoneNumber;
  #since;

  constructor(name, phoneNumber, since){
    if(name == null || phoneNumber == null || since
==null)
      throw "Invalid user"
    this.#name = name;
  }
}

```